

1. Introduction to ReactJS & Its Ecosystem

What is ReactJS?

It is a **JavaScript library** developed and maintained by **Meta** (formerly Facebook). It is specifically designed to create **optimized, dynamic user interfaces (UIs)**.

Not a Full Front-End Framework

React is only meant for creating user interfaces. To develop a complete front-end application, developers integrate React with other libraries:

- **Redux** — for managing state
- **Axios** — for backend API integrations
- **React Router** — for page routing
- **React DOM** — for managing DOM manipulations
- **Tailwind CSS** — for styling UI components
- **Jest** — for implementing unit testing

2. JSX (JavaScript XML) & Babel

What is JSX?

It is a **syntax extension in JavaScript** that allows developers to write **HTML-like code inside JavaScript** to create UI components easily.

Babel Compiler

Browsers cannot read JSX directly. A special compiler called **Babel** converts JSX code into **plain JavaScript objects** before loading it into the browser.

Integration Rules

To write JSX in standard HTML files, you must integrate CDN links for **React, ReactDOM, and Babel**, and specify:

```
type="text/babel"
```

in your script tag.

3. Vite Build Tool & Project Structure

Vite (pronounced "veet") is a modern build tool used to quickly create and run front-end applications like React.

Creation Command

`npm create vite@latest`

You can create a Vite-React app using the above terminal command.

Key Folders & Files

- **node_modules**: Holds all the packages and library dependencies
- **public**: Contains static assets like SVG files
- **src**: The main working directory where developers write JSX, JS, and CSS code
- **index.html**: The single HTML file loaded in the browser
- **package.json**: Contains brief project details, including dependencies, devDependencies, and configurations

4. React Component Architecture

What is a Component?

A component is a **reusable part of the user interface** that represents how a piece of the application looks and behaves.

Functional Components

While class components are outdated, modern React uses **functional components** — simple JavaScript functions that:

- Start with a **capital letter**
- Take optional **props** as input
- Return **JSX**

Styling Components

Local CSS files can be created for individual components and linked by importing them directly inside the JSX file.

Example:

```
import './main.css'
```